

KẾ HOẠCH PHÁT TRIỂN MÔ HÌNH

RB-LNL-Ti

Residual-Boosted Locality-iN-Locality Tiny

Mục tiêu

Cải tiến LNL-Ti theo triết lý boosting để hướng tới Top-1 accuracy $\geq 99,5\%$ trên GTSRB, trong khi mô hình cuối vẫn là một mô hình PyTorch duy nhất, có thể load và kiểm thử trực tiếp trong notebook Colab của repo gốc.

Phạm vi: kiến trúc, chiến lược huấn luyện nhiều giai đoạn, phân tích residual, ablation và đóng gói sản phẩm

Ngày lập kế hoạch: 14/07/2026

1. Tóm tắt định hướng

Hướng nghiên cứu không dùng XGBoost làm bộ phân loại cuối. Thay vào đó, mô hình học theo từng giai đoạn, trong đó mỗi giai đoạn mới tập trung sửa phần sai sót còn lại của giai đoạn trước. Cách tiếp cận này giữ được khả năng end-to-end của deep learning nhưng tiếp thu các nguyên lý quan trọng của gradient boosting: stage-wise learning, shrinkage, hard-example correction, regularization có kiểm soát và lựa chọn mô hình dựa trên validation.

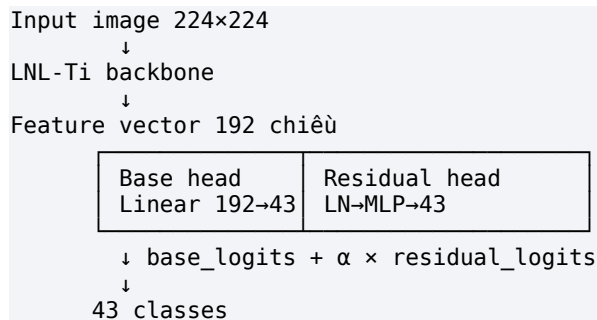
Nguyên lý trung tâm

$F_t(x) = F_{t-1}(x) + \eta f_t(x)$. Mỗi nhánh hoặc giai đoạn mới chỉ đóng vai trò hiệu chỉnh, không được phá vỡ representation và decision boundary mà mô hình nền đã học tốt.

- Mô hình nền: LNL-Ti hoặc LNL-MoEx-Ti, embedding dimension 192.
- Mô hình đề xuất: RB-LNL-Ti với base classifier và residual correction head.
- Huấn luyện theo 4 giai đoạn: representation learning, hard-example boosting, residual/confusion correction và clean calibration.
- Mô hình cuối vẫn nhận ảnh [B, 3, 224, 224] và trả logits [B, 43].
- Mục tiêu 99,5% là mục tiêu thực nghiệm, không phải cam kết trước khi huấn luyện và kiểm thử độc lập.

2. Kiến trúc RB-LNL-Ti

Phần backbone giữ lõi LNL-Ti để duy trì khả năng học locality bên trong patch và giữa các patch. Điểm thay đổi chính nằm ở classifier: bổ sung một residual head nhỏ, được huấn luyện ở giai đoạn sau để sửa các decision boundary khó.



2.1. Base classifier

```
self.head = nn.Linear(192, 43)
```

Base head giữ nhiệm vụ phân lớp chính. Nó được huấn luyện từ giai đoạn đầu cùng toàn bộ backbone và đóng vai trò F_0 trong tư duy boosting.

2.2. Residual correction head

```
self.residual_head = nn.Sequential(
    nn.LayerNorm(192),
    nn.Linear(192, 256),
    nn.GELU(),
    nn.Dropout(0.1),
    nn.Linear(256, 43),
)
self.residual_scale = nn.Parameter(torch.tensor(-2.0))

base_logits = self.head(features)
residual_logits = self.residual_head(features)
alpha = torch.sigmoid(self.residual_scale)
logits = base_logits + alpha * residual_logits
```

Khởi tạo $\text{residual_scale} = -2$ làm $\alpha \approx 0,119$. Nhánh mới chỉ tác động nhẹ ở đầu giai đoạn fine-tune, tương tự shrinkage trong boosting. Trong quá trình học, mô hình tự điều chỉnh mức đóng góp của residual head.

3. Quy trình huấn luyện nhiều giai đoạn

Giai đoạn	Epoch	Mục tiêu	Thành phần chính
Stage 1	0-89	Học representation nền	AdamW, cosine, warm-up, augmentation, MoEx, EMA
Stage 2	90-109	Sửa hard examples	Trọng số mẫu theo loss/margin; giới hạn max weight
Stage 3	110-124	Sửa residual và confusion pairs	Residual head, differential LR, pairwise margin loss
Stage 4	125-139	Clean calibration	Tắt MoEx, augmentation nhẹ, LR rất thấp, EMA

3.1. Stage 1 - Representation learning

Backbone và base head được huấn luyện end-to-end. Đây là giai đoạn tạo mô hình nền mạnh, ổn định và đủ khả năng tổng quát hóa trước khi tập trung vào các lỗi khó.

Cấu hình khởi đầu: AdamW, learning rate $3e-4$, weight decay 0,04, 5-8 epoch warm-up, cosine decay, label smoothing 0,03, drop path khoảng 0,08, EMA decay 0,9998.

MoEx được dùng với xác suất giảm dần, thay vì giữ mạnh đến cuối. Augmentation phải phù hợp biến báo: crop nhẹ, affine nhẹ, color jitter nhẹ và không horizontal flip.

3.2. Stage 2 - Phân tích residual và hard-example boosting

Sau Stage 1, chạy checkpoint EMA trên toàn bộ train và validation để lưu loss, confidence, top-1, top-2 và margin của từng ảnh. Dựa trên các chỉ số này, chia mẫu thành easy, ambiguous, hard và suspected noise.

Chỉ tăng trọng số vừa phải cho ambiguous và hard samples. Mẫu bị nghi nhiều không được boost mạnh để tránh overfit vào nhãn sai hoặc ảnh quá mờ.

3.3. Stage 3 - Residual và confusion-pair correction

Thêm residual head, freeze các block đầu và chỉ fine-tune các block cuối, normalization, base head và residual head. Learning rate của nhánh mới lớn hơn backbone nhưng vẫn bị kiểm soát.

Từ confusion matrix validation, xác định các cặp lớp thường bị nhầm và bổ sung pairwise margin loss. Không hard-code cặp lớp theo phỏng đoán; chỉ dùng các cặp được chứng minh bằng dữ liệu validation.

3.4. Stage 4 - Clean calibration

Trong 10-15 epoch cuối, tắt MoEx, random erasing và hard-example sampler; quay lại phân phối tự nhiên của dữ liệu. Giảm augmentation và label smoothing về 0 hoặc 0,01.

Mục tiêu là tối ưu clean Top-1 thay vì tiếp tục tăng robustness. Đây là bước giúp tránh việc regularization mạnh giới hạn peak accuracy.

4. Phân tích residual và gán trọng số mẫu

Mỗi ảnh cần có bản ghi phân tích để quyết định mức ưu tiên trong Stage 2. File đề xuất: `sample_difficulty.csv`.

```
image_id,true_label,predicted_label,loss,confidence,top1_prob,top2_prob,margin,difficulty_group
```

Nhóm	Tiêu chí gợi ý	Xử lý
Easy	Đúng, margin cao, loss thấp	Giữ trọng số gần 1
Ambiguous	Đúng nhưng margin thấp	Boost vừa phải
Hard	Sai hoặc loss cao	Boost mạnh hơn nhưng có giới hạn
Suspected noise	Sai lặp lại, confidence cao, ảnh bất thường	Không boost mạnh; kiểm tra thủ công nếu cần

```
w_i = clip(1 + α × normalized_loss_i, 1, w_max)
Khởi đầu: α = 1.0, w_max = 2.5
```

Cảnh báo

Không chỉ lấy toàn bộ ảnh dự đoán sai rồi oversample mạnh. Cách này dễ tập trung vào outlier, ảnh nhiễu hoặc nhãn sai và làm giảm test accuracy.

5. Hàm loss và differential learning rate

5.1. Loss tổng

```
L = L_CE + λ_pair × L_pair
```

Cross-Entropy là loss chính. Pairwise margin loss chỉ áp dụng cho các cặp confusion được xác định từ validation:

```
L_pair = max(0, margin - z_y + z_j)
Gợi ý ban đầu: margin = 0.2, λ_pair = 0.05
```

5.2. Learning rate phân tầng

Nhóm tham số	Learning rate gợi ý Stage 3
Early backbone blocks	0 - freeze
Late backbone blocks	1e-5
Base head	5e-5
Residual head	2e-4

Sự phân tầng này giúp nhánh mới học nhanh hơn nhưng không làm biến dạng representation đã được xây dựng ở Stage 1.

6. Cấu hình khởi đầu đề xuất

```
model:
  name: RB-LNL-Ti
  image_size: 224
  num_classes: 43
  embed_dim: 192
  depth: 12
  residual_hidden_dim: 256
  residual_dropout: 0.1
  residual_scale_init: -2.0

train:
  epochs: 140
  batch_size: 8
  accumulation_steps: 4
  amp: true
  optimizer: adamw
  learning_rate: 3.0e-4
  minimum_learning_rate: 1.0e-6
```

```

weight_decay: 0.04
warmup_epochs: 8
label_smoothing: 0.03
drop_path_rate: 0.08
gradient_clip_norm: 1.0

moex:
  enabled: true
  probability_start: 0.60
  probability_end: 0.00
  lambda: 0.90
  disable_last_epochs: 15

ema:
  enabled: true
  decay: 0.9998

hard_example:
  alpha: 1.0
  max_weight: 2.5

pairwise_margin:
  weight: 0.05
  margin: 0.20

```

7. Kế hoạch ablation

Không đưa toàn bộ thay đổi vào ngay lần train đầu. Mỗi module chỉ được giữ nếu giúp validation cải thiện ổn định và không làm tăng độ phức tạp vô ích.

Mã	Cấu hình	Câu hỏi cần trả lời
A0	LNL-Ti baseline	Baseline được tái tạo đúng chưa?
A1	+ training recipe + EMA	Pipeline tốt hơn giúp bao nhiêu?
A2	+ hard-example weighting	Tập trung mẫu khó có cải thiện không?
A3	+ residual head	Head tuyến tính có phải nút thắt?
A4	+ confusion-pair margin	Có sửa được các cặp lớp gần nhau không?
A5	+ clean fine-tuning	Tắt regularization cuối có tăng clean Top-1 không?
A6	+ checkpoint averaging	Có ổn định được dự đoán không?
A7	+ TTA nhẹ	Có sửa thêm các ảnh biên mà không thay checkpoint không?

8. Đánh giá và phòng tránh test leakage

- Tách validation từ train theo group/sequence nếu metadata cho phép; tránh chia ngẫu nhiên các frame gần giống nhau.
- Chỉ dùng validation để chọn hyperparameter, checkpoint, hard-sample weights và confusion pairs.
- Official test set chỉ được chạy sau khi chốt cấu hình.
- Không chọn seed, TTA hoặc checkpoint dựa trực tiếp trên official test.
- Báo cáo riêng single-view Top-1 và TTA Top-1 nếu có sử dụng TTA.
- Để đạt 99,5% trên 12.630 ảnh test, mô hình chỉ được sai khoảng 63 ảnh.

9. Yêu cầu plug-and-play

Mô hình cuối phải giữ interface đơn giản để notebook kiểm chứng chỉ cần import model, load state_dict và gọi model(images).

```
from rb_lnl_ti import RB_LNL_Ti

model = RB_LNL_Ti(num_classes=43)
state = torch.load('rb_lnl_ti_gtsrb.pth', map_location='cpu')
model.load_state_dict(state)
model.eval()

logits = model(images) # [B, 43]
```

- Input: tensor [B, 3, 224, 224].
- Output: logits [B, 43].
- Không cần sklearn hoặc XGBoost khi inference.
- Không bắt buộc truyền swap_index khi test.
- Checkpoint nộp nên là state_dict thuần để giảm phụ thuộc phiên bản.

10. Sản phẩm cần lưu

```
submission/
├── rb_lnl_ti.py
├── rb_lnl_ti_gtsrb.pth
├── Instructions_RB_LNL_Ti.ipynb
├── requirements.txt
├── README.md
├── config.yaml
├── results/
│   ├── test_result.png
│   ├── metrics.json
│   ├── predictions.csv
│   ├── confusion_matrix.csv
│   └── sample_difficulty.csv
```

11. Lộ trình triển khai

Mốc	Công việc	Điều kiện hoàn thành
M1	Tái tạo baseline LNL-Ti/LNL-MoEx-Ti	Train, validation, checkpoint và test chạy ổn định
M2	Thêm logging residual	Có sample_difficulty.csv và confusion matrix
M3	Triển khai hard-example boosting	Có ablation A2 so với A1
M4	Triển khai residual head	Có ablation A3 và kiểm tra α
M5	Thêm confusion-pair margin	Có ablation A4, không hard-code cặp lớp
M6	Clean fine-tuning và EMA	Có best single-view checkpoint
M7	Checkpoint averaging/TTA	Chỉ giữ nếu validation hoặc protocol độc lập xác nhận
M8	Đóng gói bài nộp	Notebook sạch load được checkpoint và tái tạo kết quả

12. Rủi ro và biện pháp kiểm soát

Rủi ro	Biện pháp
Hard-example weighting overfit vào nhiễu	Clip trọng số; tách suspected noise; quay lại natural

	sampling ở cuối
Residual head phá base classifier	Khởi tạo scale nhỏ; differential LR; freeze early blocks
Pairwise loss quá mạnh	Bắt đầu $\lambda=0,05$; chỉ dùng cặp confusion từ validation
MoEx giới hạn peak clean accuracy	Giảm xác suất dẫn và tắt trong 15 epoch cuối
Validation leakage do frame gần giống	Group-aware split theo track/sequence nếu có
RTX 2050 thiếu VRAM	Batch 8 + accumulation 4 + AMP; dùng Colab cho train cuối
Mục tiêu 99,5% không đạt	Giữ ablation minh bạch; cân nhắc LNL-MoEx-S nếu Tiny dùng dưới ngưỡng

13. Kết luận

RB-LNL-Ti là hướng cải tiến LNL-Ti dựa trên tư duy boosting nhưng không đưa XGBoost vào sản phẩm cuối. Giá trị cốt lõi nằm ở việc huấn luyện theo từng stage, đo residual thật, tập trung có kiểm soát vào hard samples, bổ sung residual classifier với shrinkage và quay lại clean fine-tuning ở cuối.

Hướng này có ưu điểm là giải thích được từng cải tiến thông qua ablation, giữ mô hình cuối gọn và plug-and-play, đồng thời phù hợp với mục tiêu nghiên cứu vượt kết quả LNL-Ti gốc. Mốc Top-1 $\geq 99,5\%$ chỉ được xác nhận khi checkpoint cuối đạt kết quả đó trên official GTSRB test theo protocol độc lập và tái lập được.

Quyết định thiết kế

Ưu tiên sửa residual có bằng chứng hơn là tăng kích thước mô hình hoặc thêm module theo trực giác. Mỗi thay đổi phải trả lời được: nó đang sửa lỗi nào, và validation có chứng minh điều đó hay không?